

Proposta de Arquitetura Técnica — Aderência à Guideline de TI da EMS IT

Preparado para: Jabil **Elaborado por:** Victor Lucas — Engenheiro Fullstack Sênior **Data:** 14/07/2026 (revisão 5 — itens 16, 17 e 18 implementados e mesclados na branch principal; ver seções 8, 9 e 10)

Sumário executivo

Este documento apresenta a arquitetura do projeto de referência **Bil** — um **dashboard de OEE (Overall Equipment Effectiveness) para linha de produção SMT** da Jabil, com autenticação de usuários e acompanhamento de paradas (downtime) — organizada **item a item conforme a Guideline de TI da EMS IT**, no mesmo formato usado pela Jabil para avaliar a proposta original (Item da Guideline → Arquitetura Proposta → Aderência → Adequações).

A revisão anterior deste documento foi avaliada pela Jabil e recebeu um parecer com 6 itens em "Não atende" e 6 em "Parcial", de 18 itens avaliados. Esta revisão incorpora o plano de adequação de cada um desses itens diretamente na arquitetura proposta, para que o documento sirva como a proposta final a ser submetida — não mais como um registro do que já existe no repositório hoje.

Como ler a tabela abaixo: a coluna "Aderência" reflete o **compromisso da proposta ajustada**, não necessariamente o que já está codificado neste momento no repositório. Onde a implementação ainda não foi feita, isso é dito explicitamente na coluna "Plano de implementação", com a tecnologia concreta escolhida — não uma recomendação genérica.

1. Aderência à Guideline de TI da EMS IT

#	Item da Guideline EMS IT	Arquitetura Proposta	Aderência	Plano de implementação
1	Arquitetura em camadas	Clean Architecture + DDD tático (core / domain / infra)	Atende	Já implementado no backend (backend/src/core, backend/src/domain, backend/src/infra). Ver detalhe na seção 2.1.
2	Separação de responsabilidades	Core (genérico) / Domain (regra de negócio) / Infra (detalhe técnico)	Atende	Regra: dependência sempre aponta para dentro — domain não conhece Prisma, Redis ou NestJS, só interfaces. Ver seção 2.1.
3	Dependency Injection	NestJS DI nativo	Atende	Container de DI do próprio framework, sem biblioteca adicional. Ver seção 2.2.
4	Padrões de Projeto (GoF)	Repository, Factory, Strategy, Adapter	Atende	UsersRepository (Repository), Encrypter/HashComparer (Strategy),

				Uploader/CacheRepository (Adapter). Ver seção 2.2.
5	Qualidade de código	Vitest + ESLint + Prettier + SonarQube/SonarCloud	Atende (com adequação)	Manter Vitest/ESLint/Prettier e adicionar SonarQube/SonarCloud com Quality Gate (cobertura mínima, complexidade ciclomática, duplicação de código) bloqueando merge. Ver seção 3.
6	SonarQube obrigatório	SonarQube/SonarCloud integrado ao pipeline	Atende (com adequação)	Integração ao Azure DevOps Pipeline com Quality Gate obrigatório antes do merge para a branch principal. Ver seção 3.
7	Logging corporativo	Winston (ou Pino) com log estruturado	Atende (com adequação)	Substituir o ConsoleLogger padrão do Nest por logging estruturado com níveis, Correlation ID/Request ID e integração com a ferramenta de monitoramento da Jabil. Ver seção 4.
8	Tratamento de exceções	Either Pattern (regra de negócio) + Global Exception Filter (exceção inesperada)	Atende (com adequação)	Either já implementado para erro de negócio; adicionar GlobalExceptionHandler do NestJS para exceção não tratada, com resposta HTTP padronizada. Ver seção 2.3.
9	Internacionalização (i18n)	react-i18next (frontend) + Accept-Language (backend)	Atende (com adequação)	Externalizar toda mensagem/validação para arquivo de recurso. Ver seção 5.
10	Documentação XML	Não aplicável a stack Node/TypeScript	N/A — equivalente proposto	TypeDoc/JSDoc + OpenAPI (Swagger) como equivalente funcional ao padrão de documentação XML do .NET. Ver seção 6.
11	Convenções de nomenclatura	PascalCase / camelCase	Atende	Já aplicado em todo o código (classes em PascalCase,

				variáveis/métodos em camelCase).
12	Banco ACID	PostgreSQL 17	Atende	Já implementado via Prisma ORM. Ver seção 2.1.
13	CI/CD	Pipeline completo no Azure DevOps	Atende (com adequação)	Build → Testes → SonarQube → Dependency/Security Scan → artefato versionado → aprovação de release → deploy DEV/QA/UAT/Produção. Ver seção 7.
14	Code Review	Pull Request obrigatório antes do merge	Atende	Já previsto no fluxo de trabalho da equipe.
15	Testes automatizados	Unit (Vitest) + E2E (Playwright/Vitest e2e)	Atende	Já implementado — use cases testados com repositórios fake em memória, sem depender de banco/infra. Ver seção 2.1.
16	Observabilidade	OpenTelemetry + Prometheus + Grafana	Atende	Health checks, métricas e distributed tracing implementados e mesclados na master. Ver seção 8.
17	Auditoria	Audit Trail dedicado	Atende	Tabela audit_logs implementada, alimentada por eventos de domínio. Mesclado na master. Ver seção 9.
18	Segurança	JWT RS256 + RBAC + Rate Limiting + Helmet/CORS + OAuth2/OIDC + Key Vault	Atende (parcial) — pendências com a Jabil	RBAC, rate limiting e headers OWASP implementados. OAuth2/OIDC, cofre de segredos e criptografia em repouso dependem de decisão/infra da Jabil. Ver seção 10.

Resultado consolidado: dos 18 itens, **9 já atendem hoje sem nenhuma adequação pendente** (itens 1, 2, 3, 4, 11, 12, 14, 15, 16, 17 — arquitetura, DI, padrões de projeto, nomenclatura, banco, code review, testes, observabilidade e auditoria) e **8 passam a atender com a adequação descrita** nas seções técnicas abaixo (itens 5, 6, 7, 8, 9, 13, 18), restando **1 item não aplicável à stack** (documentação XML, item 10) com equivalente proposto. O item 18 (Segurança) já tem a parte que independe da Jabil implementada; a parte restante aguarda resposta da Jabil às perguntas da seção 10.1.

2. Backend — o que já atende hoje

2.1 Arquitetura em camadas, separação de responsabilidades e banco ACID (itens 1, 2, 12)

Usamos **Clean Architecture + DDD tático**:

```
infra → domain → core
(detalhe) (regra) (genérico)
```

As dependências apontam sempre para dentro. O núcleo de regras de negócio não sabe que Postgres, Redis ou NestJS existem — ele só conhece interfaces (contratos).

Fisicamente no repositório: `backend/src/core` , `backend/src/domain` , `backend/src/infra` .

Área	Tecnologia	Versão instalada	Observação
Framework	NestJS	10.x	Módulos, DI e decorators nativos
Linguagem	TypeScript	5.1	target: es2022, strict: true
ORM / Banco	Prisma 5.2 + PostgreSQL 17	Banco ACID (item 12) já implementado	Schema hoje tem a tabela users; domínio de OEE/downtime a modelar
Cache	Redis 7 + ioredis 5.3	RedisCacheRepository implementa interface CacheRepository genérica — cache-aside, TTL 15 min	
Autenticação	JWT RS256 via @nestjs/jwt + passport-jwt	Chave assimétrica, guard global (JwtAuthGuard)	Ver adequação de segurança na seção 10
Validação	Zod v3 + zod-validation-error	Valida env e payload do JWT	
Upload/Storage	Storage local em disco atrás de interface Uploader	Trocável por S3/R2 sem tocar em regra de negócio	
Testes	Vitest 0.34 (unit) + config separada e2e	Item 15 (testes automatizados)	
Package manager	pnpm via Turborepo 2.5	Monorepo backend + frontend	

Ganho concreto (itens 1, 2 e 15 juntos): testes de use case usam repositórios fake em memória (`test/repositories`), rodam em segundos, sem depender de banco ou Redis.

2.2 Dependency Injection e Padrões de Projeto — GoF (itens 3, 4)

```
@Injectable()
export class AuthenticateUserUseCase {
  constructor(
    private usersRepository: UsersRepository, // Repository – interface, não
    implementação
    private hashComparer: HashComparer,      // Strategy
    private encrypter: Encrypter,            // Strategy
  ) {}

  async execute({ email, password }: AuthenticateUserUseCaseRequest) {
    const user = await this.usersRepository.findByEmail(email)
    if (!user) return left(new WrongCredentialsError())

    const isPasswordValid = await this.hashComparer.compare(password, user.password)
    if (!isPasswordValid) return left(new WrongCredentialsError())

    const accessToken = await this.encrypter.encrypt({ sub: user.id.toString() })
    return right({ accessToken })
  }
}
```

`UsersRepository` é uma classe abstrata definida no `domain` (padrão **Repository**).
`HashComparer` / `Encrypter` são interfaces de estratégia (padrão **Strategy**), injetadas via DI nativo do NestJS. Quem implementa de fato (Prisma em produção, fake em memória em teste) mora em `infra / test/repositories` — o use case nunca importa Prisma. `Uploader` (storage) e `CacheRepository` (Redis) seguem o mesmo molde de **Adapter**: a regra de negócio depende só da interface.

2.3 Tratamento de exceções (item 8)

Use cases retornam um tipo `Either<Erro, Sucesso>` (`backend/src/core/either.ts`) em vez de usar `throw` para erro de regra de negócio:

```
export class Left<L, R> {
  constructor(readonly value: L) {}
  isLeft(): this is Left<L, R> { return true }
  isRight(): this is Right<L, R> { return false }
}

export class Right<L, R> {
  constructor(readonly value: R) {}
  isLeft(): this is Left<L, R> { return false }
  isRight(): this is Right<L, R> { return true }
}

export type Either<L, R> = Left<L, R> | Right<L, R>
```

Por quê: o próprio tipo de retorno do use case documenta todo erro possível daquela operação. O TypeScript obriga quem chama a checar `isLeft()` antes de usar o valor.

Adequação para atender integralmente o item 8: o Either cobre erro de negócio esperado; falta o tratamento padronizado de exceção *inesperada* (falha de infra, bug). Proposta: um `GlobalExceptionHandler` (`@Catch()` do NestJS) capturando qualquer exceção não tratada, padronizando o corpo de resposta HTTP (código, mensagem amigável, `traceId`), logando automaticamente o stack trace e nunca vazando detalhe interno para o cliente. O Either continua sendo a forma de modelar erro de negócio — o filtro global é a rede de segurança para o que escapar dele.

2.4 Eventos de domínio

A infraestrutura de eventos de domínio (`core/events/domain-events.ts` , `AggregateRoot.addDomainEvent`) já existe no `core` , pronta para quando o domínio de OEE precisar — por exemplo, notificar alguém quando uma parada de linha ultrapassar um limite de tempo, ou alimentar o Audit Trail (seção 9) como consumidor de eventos.

3. Qualidade de código e SonarQube (itens 5 e 6)

Hoje: Vitest (testes), ESLint (lint) e Prettier (formatação), sem análise estática de qualidade/segurança.

Adequação proposta: integrar **SonarQube (ou SonarCloud)** ao pipeline de CI/CD (seção 7), com Quality Gate cobrindo:

- Cobertura mínima de testes (definir percentual-alvo com a Jabil, sugestão inicial: 80% em `domain / core`);
- Complexidade ciclomática por método/função;
- Duplicação de código;
- SAST (Static Application Security Testing) — vulnerabilidades conhecidas no próprio código;
- Bloqueio obrigatório de merge para a branch principal enquanto o Quality Gate não passar.

4. Logging corporativo (item 7)

Hoje o backend usa o `ConsoleLogger` padrão do NestJS (log não estruturado, sem correlação entre requisições).

Adequação proposta:

- Logging estruturado com **Winston** ou **Pino** (formato JSON, pronto para ingestão por ferramenta corporativa);
- Níveis padronizados: `Information` , `Warning` , `Error` , `Debug` ;
- **Correlation ID** e **Request ID** gerados por requisição (middleware) e propagados em toda a cadeia de log daquela chamada;
- Rastreamento de exceções (integrado ao `GlobalExceptionHandler` da seção 2.3);
- Centralização e integração com a solução de monitoramento corporativa da Jabil — **Azure Monitor**, CloudWatch ou ELK, a confirmar qual já está homologada na Jabil.

5. Internacionalização — i18n (item 9)

Hoje as telas e mensagens de validação estão hardcoded em PT-BR, sem camada de i18n.

Adequação proposta:

- Frontend: **react-i18next**, com todo texto de UI migrado de string literal para chave de tradução em arquivo de recurso (`pt-BR.json` , `en-US.json` conforme necessidade da Jabil);
- Backend: suporte a header `Accept-Language` , com mensagens de erro/validação (hoje geradas pelo Zod) também externalizadas para arquivo de recurso, em vez de string fixa no código.

6. Documentação (item 10 — N/A, equivalente proposto)

O padrão de documentação XML da guideline é específico do ecossistema .NET e não se aplica a uma stack TypeScript/NestJS. Equivalente funcional proposto:

- **TypeDoc/JSdoc** para documentação de código (classes, use cases, interfaces de domínio);
- **OpenAPI (Swagger)** gerado a partir dos controllers NestJS, documentando todo endpoint HTTP exposto — contrato de request/response, códigos de erro e autenticação necessária.

7. CI/CD (item 13)

Hoje não existe pipeline configurado no repositório.

Adequação proposta — pipeline no Azure DevOps (ferramenta já usada pela EMS IT, substituindo a alternativa GitHub Actions cogitada anteriormente):

1. **Build** — backend e frontend via Turborepo;
2. **Testes automatizados** — unit + e2e (Vitest/Playwright);
3. **SonarQube** — Quality Gate (seção 3);
4. **Dependency Scan / Security Scan (SAST)**;
5. **Geração de artefato versionado**;
6. **Aprovação manual de release** (gate por ambiente);
7. **Deploy automatizado** entre os ambientes **DEV** → **QA** → **UAT** → **Produção**.

8. Observabilidade (item 16) — implementado

Implementado e mesclado na `master` (`backend/src/infra/observability/`):

- **Health Checks** — `GET /health` (rota pública, via `@nestjs/terminus`), verificando Postgres e Redis; retorna `up / down` por dependência, pronto para probe de orquestrador (Kubernetes, Azure App Service, etc.);
- **Métricas** — `GET /metrics` (rota pública, formato Prometheus), com métricas padrão do processo Node.js e métricas HTTP customizadas (`http_requests_total` , `http_request_duration_seconds`) segmentadas por rota, método e status code;
- **Distributed Tracing** — **OpenTelemetry**, instrumentação automática (HTTP, Express, Prisma, Redis) carregada antes do bootstrap da aplicação; exporta para o console em desenvolvimento e para um collector OTLP (Grafana Tempo, Jaeger, Azure Monitor, etc.) quando a variável `OTEL_EXPORTER_OTLP_ENDPOINT` está configurada;
- **Dashboards** — **Prometheus + Grafana** sobem junto com a infraestrutura de desenvolvimento (`docker-compose.yml`), com scrape config e um dashboard (`observability/grafana/dashboards/bil-backend.json`) já provisionados automaticamente: requisições por segundo, latência p95, taxa de erro 5xx e event loop lag.

Pendência com a Jabil: o destino de produção das métricas/traces (Prometheus+Grafana self-hosted vs. Azure Monitor/Application Insights corporativo) depende de qual ferramenta de observabilidade a Jabil já tem homologada — ver pergunta na seção 10.1.

9. Auditoria (item 17) — implementado

Implementado e mesclado na `master`: tabela dedicada `audit_logs` (`backend/src/core/audit/` , `backend/prisma/schema.prisma`), separada do log de aplicação, registrando por evento:

- Usuário que executou a ação (`actorId` , quando aplicável);
- Data/hora da operação (`createdAt`);
- Operação executada (`action` , ex.: `user.registered` , `user.login_succeeded` , `user.login_failed`);
- Dado alterado, no formato **Before/After** (`before` / `after` , JSON);
- Origem da requisição (`ipAddress` , `userAgent`).

Como planejado, o trail nasce como **consumidor de eventos de domínio** (`DomainEvents` , seção 2.4):

`User` agora é um `AggregateRoot` que dispara `UserRegisteredEvent` na criação da conta, e o `subscriber OnUserRegistered` grava o registro de auditoria a partir desse evento — o use case de registro de usuário não sabe que auditoria existe. Tentativas de login (sucesso e falha) são auditadas diretamente no controller de autenticação, já que precisam do contexto HTTP (IP, user agent) que não pertence à camada de domínio — isso também cobre o pedido de auditoria de autenticação do item 18. Consulta via `GET /audit-logs` (autenticado, paginado).

Ponto em aberto: o domínio de OEE/downtime ainda não existe no código (seção 12); quando for modelado, a mesma infraestrutura de `DomainEvents` → `AuditLogRepository` se aplica a operações como "resolução de evento de parada", sem trabalho adicional de infraestrutura.

10. Segurança (item 18) — implementado em parte, restante pendente da Jabil

Base: autenticação via JWT **RS256** com chave assimétrica e guard global (`JwtAuthGuard`), com rotas públicas via decorator `@Public()` .

O que já está implementado e mesclado na `master` :

- **RBAC (Role-Based Access Control):** `User` ganhou um `role` (`OPERATOR` / `SUPERVISOR` / `ADMIN` , default `OPERATOR`), propagado no JWT. Decorator `@Roles()` + `RolesGuard` (guard global, executa depois do `JwtAuthGuard`) bloqueiam rota sem o papel exigido — hoje demonstrado em `GET /accounts` , restrito a `ADMIN` ;
- **Rate Limiting:** `@nestjs/throttler` — limite global de 100 req/min e limite mais restritivo de 5 req/min em `POST /sessions` (login), mitigando força bruta;
- **Proteção OWASP Top 10 (parcial):** validação de entrada via `Zod` (já existia); `helmet()` adicionado no bootstrap (headers como `Strict-Transport-Security` , `X-Frame-Options` , `X-Content-Type-Options`); CORS configurável por ambiente via `CORS_ORIGIN` ;
- **Auditoria de autenticação:** tentativas de login (sucesso e falha) já alimentam o Audit Trail da seção 9, com IP e user agent da requisição.

O que segue pendente — depende de decisão/infraestrutura da Jabil:

- **OAuth2/OpenID Connect** com IdP corporativo, para SSO;
- **Gerenciamento seguro de segredos** (Azure Key Vault / AWS Secrets Manager) para `JWT_PRIVATE_KEY` / `JWT_PUBLIC_KEY` e credenciais de banco, hoje em variável de ambiente;
- **Criptografia de dados sensíveis em repouso** — depende de quais campos do domínio de OEE/downtime (seção 12, ainda não modelado) a Jabil classifica como sensíveis;

- **Proteção contra IDOR** nas rotas de OEE/downtime — também depende desse domínio existir.

10.1 Perguntas para a Jabil

1. **Identidade corporativa:** federar login com IdP corporativo (Azure AD/Entra ID, Okta, outro) via OAuth2/OIDC, ou o login local (email/senha) atual é suficiente para este projeto? Se for federar, qual provedor, e quem configura o app registration do lado da Jabil?
2. **Cofre de segredos:** a infraestrutura da Jabil já disponibiliza Azure Key Vault, AWS Secrets Manager ou equivalente para este projeto, ou isso ainda precisa ser provisionado?
3. **Papéis de negócio:** além de um papel genérico de administrador, quais são os papéis reais esperados no domínio de OEE/downtime (ex.: operador de linha só reporta paradas da própria linha, supervisor aprova/edita, engenharia configura metas de OEE)? Isso define a granularidade real do RBAC.
4. **CORS em produção:** quais domínios vão servir o frontend em cada ambiente (DEV/QA/UAT/Produção)? Necessário para configurar `CORS_ORIGIN` restritivamente — hoje o default é permissivo, só para não travar o desenvolvimento local.
5. **Dados sensíveis em repouso:** quando o domínio de OEE/downtime for modelado, quais campos a Jabil considera sensíveis o suficiente para exigir criptografia em repouso (dados de produção proprietários, PII de operador, etc.)?
6. **Ferramenta de observabilidade corporativa** (relacionado à seção 8): a Jabil já tem Azure Monitor/Application Insights homologado, ou o time prefere que o projeto suba sua própria stack Prometheus + Grafana em produção?

11. Frontend — stack de apoio

Para completude, a stack de frontend (não avaliada item a item pela guideline, mas que sustenta as telas que consomem o backend acima):

Área	Tecnologia	Versão instalada
Framework	React	18.2
Build tool	Vite	5.0
Linguagem	TypeScript	5.2
Roteamento	React Router DOM	6.21
Estado de servidor	TanStack Query	5.17
Formulários	react-hook-form 7.49 + Zod v3	
Design system	shadcn/ui sobre Radix UI	Componentes copiados para o repo, não pacote fechado
Estilo	Tailwind CSS 3.3	
Testes	Vitest 1.2 + Testing Library + Playwright 1.41 (e2e)	
Mock de API	MirageJS 0.1.48	Única fonte de dados das telas de OEE/downtime até o backend de domínio existir

Todo dado vindo da API passa por `useQuery` / `useMutation` do TanStack Query — nunca em `useState` global ou Context solto. Cliente HTTP único: `bilApi` (`frontend/src/lib/bil-api.ts`), instância `axios` com `withCredentials: true` .

Telas hoje implementadas: **Dashboard** (KPIs de OEE, tendência, composição, Pareto de paradas), **Downtime** (histórico e resolução de paradas) e **Auth** (login/cadastro).

12. Itens de domínio ainda em aberto (fora do escopo da guideline de TI)

Independente da aderência à guideline, dois pontos de domínio seguem em aberto e não devem ser confundidos com os itens 1–18 acima:

1. **Backend do domínio de OEE/downtime não existe ainda.** O schema Prisma só tem `users` ; as métricas de OEE e o histórico de downtime hoje vêm do mock MirageJS. É o próximo módulo de domínio a modelar (linha, equipamento, evento de parada, cálculo $OEE = disponibilidade \times performance \times qualidade$).
 2. **Storage em disco local.** A interface `Uploader` já isola isso — migrar para S3/R2 é só trocar a implementação, sem tocar em use case, caso seja necessário anexar evidência/foto a um evento de parada.
-

Conclusão

Com as adequações descritas nas seções 3 a 10, a proposta passa a atender **17 dos 18 itens da Guideline de TI da EMS IT** (o único item fora do escopo direto, documentação XML, tem equivalente funcional proposto na seção 6). Os itens 16 (Observabilidade) e 17 (Auditoria) já estão **implementados e mesclados na branch principal**, não mais como plano — health checks, métricas, tracing e Audit Trail funcionando de ponta a ponta. O item 18 (Segurança) está **implementado na parte que independe da Jabil** (RBAC, rate limiting, headers OWASP, auditoria de autenticação); a parte restante (SSO corporativo, cofre de segredos, criptografia em repouso) aguarda as respostas às perguntas da seção 10.1. Os demais itens que exigem adequação (5, 6, 7, 9, 13) têm tecnologia e abordagem concretas definidas acima — não pendências em aberto sem direção. Pontos que dependem de decisão conjunta com a Jabil estão consolidados nas perguntas da seção 10.1.